



MANIPULAÇÃO DE GRAFOS DINÂMICOS COM LISTAS LIGADAS

Nuno Miguel Fernandes Oliveira

Nº 31550 – Regime Pós-laboral

Ano letivo 2024/2025

Licenciatura em Engenharia de Sistemas Informáticos

**Escola Superior de Tecnologia
Instituto Politécnico do Cávado e do Ave**

Manipulação de Grafos com Listas Dinâmicas
Nuno Oliveira

Índice

Resumo	4
1. Introdução	5
1.1. Contexto do Projeto.....	5
1.2. Interpretação do problema.....	5
1.3. Objetivos	5
1.4. Metodologia.....	5
2. Enquadramento Teórico	6
2.1. Estruturas	6
2.2. Listas Ligadas e Grafos	7
2.3. Funções.....	7
2.3.1. Criar Antena	7
2.3.2. Inserir Antena.....	7
2.3.3. Criar Aresta.....	8
2.3.4. Inserir Aresta	8
2.3.5. Calcular Arestas	8
2.3.6. Ligar Arestas às Antenas Origem	8
2.3.7. Ligar Arestas às Antenas Destino.....	8
2.3.8. Remover Antena.....	9
2.3.9. Ler Ficheiro	9
2.3.10. Gravar Ficheiro	9
2.3.11. Gravar Ficheiro Binário.....	9
2.3.12. Ler Ficheiro Binário	9
2.3.13. Procurar Antena Pelo Identificador	10
2.3.14. Reset aos Visitados.....	10
2.3.15. Número de Antenas com a Mesma Frequência	10
2.3.16. Depth First Traversal (Travessia em Profundidade)	10
2.3.17. Breadth First Traversal	10
2.3.18. Todos Caminhos Possíveis	11

Manipulação de Grafos com Listas Dinâmicas
Nuno Oliveira

3. Implementação	11
3.1. Percurso até à solução.....	11
3.3. Leitura e Escrita em Ficheiros.....	12
3.3.1. Ficheiros Texto.....	12
3.3.2. Ficheiros Binários.....	12
3.4. Travessias DFT e BFT.....	13
3.4.1. Depth First Traversal (DFT).....	13
3.4.2. Breadth First Traversal (BFT)	14
Conclusão.....	15

Resumo

Este projeto, centrado na manipulação de grafos dinâmicos com recurso a listas ligadas, tem como objetivo aprofundar os conhecimentos adquiridos na unidade curricular de Estruturas de Dados Avançadas. Partindo da primeira fase, que abordava listas simples para representação de antenas, nesta nova fase o foco é a ligação entre as antenas que partilham a mesma frequência via aresta, simulando um grafo dinâmico.

Através de estruturas dinâmicas, foram implementadas funcionalidades como criação, inserção, ligação e remoção de antenas e arestas, bem como algoritmos de travessia em profundidade (DFT) e largura (BFT), promovendo uma abordagem eficiente, promovendo uma gestão dinâmica do grafo.

1. Introdução

1.1. Contexto do Projeto

Na sequência da primeira fase do projeto da unidade curricular de Estruturas de Dados Avançadas, a segunda fase propôs a implementação de um grafo dinâmico, composto por antenas interligadas através de arestas. Cada antena representa um vértice do grafo, e as ligações entre antenas da mesma frequência representam as arestas.

1.2. Interpretação do problema

Considera-se uma rede de antenas com localizações e frequências específicas. Quando duas antenas partilham a mesma frequência, devem ser conectadas por uma aresta. O objetivo é construir dinamicamente um grafo e permitir operações de análise e manipulação, como travessias e remoções.

1.3. Objetivos

Esta fase pretende aplicar os conceitos de estruturas de dados dinâmicas à criação de um grafo, implementado com listas ligadas. Entre os objetivos estão: ligação de vértices (antenas), gestão das arestas, leitura e escrita do grafo em ficheiros binários e aplicação de algoritmos de travessia em profundidade e em largura.

1.4. Metodologia

A metodologia adotada baseou-se na criação de estruturas de dados dinâmicas via listas ligadas, representando vértices e arestas, com ordenação por coordenadas. Posteriormente, implementaram-se funções de ligação, cálculo e organização das arestas, bem como leitura, gravação do grafo em ficheiros binários e algoritmos de travessia.

2. Enquadramento Teórico

2.1. Estruturas

No desenvolvimento do projeto, foram utilizadas duas estruturas principais: Antena e Aresta. Estas estruturas são fundamentais para a construção e navegação do grafo. A separação clara entre vértices e arestas permite um modelo eficiente e modular.

A estrutura Antena (*Figura I*) representa um vértice do grafo. Abrange dados como:

- **id:** Identificador único da antena;
- **linha, coluna:** Coordenadas das Antenas na matriz;
- **tipoAntena:** Frequência da Antena representada por um caracter;
- **visitado:** Controlo para travessias;
- **proxAntena:** Apontador para a próxima antena na lista ligada;
- **arestas:** Apontador para a lista ligada de arestas, cujo esta antena é a origem das mesmas.
-

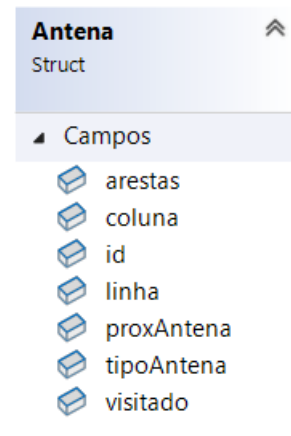


Figura I

A estrutura Aresta (*Figura II*) representa uma ligação entre duas Antenas com a mesma frequência. Abarca os seguintes dados:

- **destinoID:** Identificador da Antena destino.
- **origem:** Apontador para a Antena que originou a Aresta;
- **destino:** Apontador para a Antena destino;
- **distancia:** Peso entre as Antenas;
- **proxAresta:** Apontador para a próxima Aresta na lista.

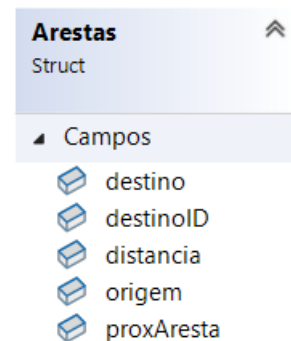


Figura II

2.2. Listas Ligadas e Grafos

Assim como na Fase I, a base do projeto assenta nas Listas Ligadas, em que cada elemento da lista referencia o seguinte. Todavia, na Fase II, o conceito é expandido para a representação de Grafos. Assim sendo, a base do projeto fica com duas características importantes:

- A lista principal contém todas as Antenas.
- Cada Antena detém a lista de arestas de onde as mesmas são a origem, apontando para outras antenas que partilham a mesma frequência, que são o destino.

Decidiu-se utilizar esta abordagem para garantir as seguintes vantagens:

- Adição e Remoção de Antenas;
- Adição e Remoção de Arestas.

Com isto, garante-se o armazenamento dinâmico do Grafo.

2.3. Funções

Os propósitos das funções:

2.3.1. Criar Antena

Antena* CriarAntena(**int** linha, **int** coluna, **char** tipoAntena, **int*** id, **int*** val);

Aloca de memória de uma antena e preenche os seus campos iniciais, passados por parâmetros.

2.3.2. Inserir Antena

Antena* InserirAntena(**Antena*** head, **Antena*** nova, **int*** val);

Insere a antena numa lista ligada ordenada com base nas coordenadas, dando prioridade à linha e de seguida à coluna.

2.3.3. Criar Aresta

Arestas* CriarAresta(**Antena*** origem, **int** destinoID, **int** distancia, **int*** val);

Cria uma aresta entre uma Antena de origem e um destino identificado por ID. O peso da Aresta é passado como parâmetro. A ligação à Antena de destino real é feita posteriormente.

2.3.4. Inserir Aresta

Arestas* InserirAresta(**Arestas*** head, **Arestas*** nova, **int*** val);

Insere uma nova Aresta na lista das Arestas, ordenando com base nas coordenadas da Antena origem e distância.

2.3.5. Calcular Arestas

Arestas* CalcularArestas(**Antena*** antenaHead, **Arestas*** arestasHead, **int*** val);

Percorre a lista de Antenas à procura de todas as combinações possíveis das mesmas. Se ambas partilharem a mesma frequência, é criada uma Aresta com base na distância entre as duas Antenas.

2.3.6. Ligar Arestas às Antenas Origem

Antena* LigarAntenasArestas(**Antena*** antenaHead, **Arestas*** arestasHead, **int*** val);

Associa todas as Arestas à sua Antena de origem. Esta função garante que cada aresta pertence corretamente à lista de Arestas da Antena que a originou.

2.3.7. Ligar Arestas às Antenas Destino

Antena* LigarAntenaDestinoAntenaId(**Antena*** antenahead, **int*** val);

Percorre todas as Arestas das Antenas origem e associa a Antena destino pelo campo da Aresta ao destinoID. Esta ligação só é possível depois de carregar o Grafo.

2.3.8. Remover Antena

Antena* RemoverAntena(**Antena*** antenaHead, **int** antenaID, **int*** val);

Remove a Antena com determinado ID, recebido por parâmetro, e todas as Arestas associadas a ela. Além disso, remove todas as Arestas de outras Antenas que a têm como destino.

2.3.9. Ler Ficheiro

Antena* LerFicheiro(**char*** nomeFicheiro, **Antena*** antenaHead, **int*** id, **int*** val);

Lê um ficheiro de texto com a representação em matriz das Antenas. Quando o carácter diferente de '.' é criada uma Antena e Inserida na respetiva lista.

2.3.10. Gravar Ficheiro

int GravarFicheiro(**char*** nomeFicheiro, **Antena*** antenaHead);

Escreve de forma matricial as antenas num ficheiro de texto. Preenche com '.' em todas as coordenadas que não correspondem às Antenas.

2.3.11. Gravar Ficheiro Binário

int GravarFicheiroBinario(**char*** nomeFicheiro, **Antena*** antenaHead, **int** id);

Guarda a estrutura do Grafo num ficheiro binário, utilizando estruturas auxiliares sem apontadores. Cada Antena é seguida pelas respetivas Arestas.

2.3.12. Ler Ficheiro Binário

Antena* LerFicheiroBinario(**char*** nomeFicheiro, **int*** id, **int*** val)

Reconstrói o Grafo a partir de um ficheiro binário. Após ler os dados, liga as Arestas às respetivas Antenas de origem.

2.3.13. Procurar Antena Pelo Identificador

Antena* ProcurarAntenaPeloID(**Antena*** antenaHead, **int** antenaID);

Função auxiliar para encontrar uma antena específica com base no seu número identificador. Usada, por exemplo, nos algoritmos de travessia.

2.3.14. Reset aos Visitados

bool ResetarVisitados(**Antena*** antenaHead);

Percorre todas as antenas e altera o valor do campo visitado para zero. Essencial para garantir o sucesso na realização dos algoritmos de travessias.

2.3.15. Número de Antenas com a Mesma Frequência

int NumeroAntenasMesmaFrequencia(**Antena*** antenaHead, **Antena*** atual);

Devolve o número de Antenas existentes no Grafo com a mesma frequência de uma antena de referência. Auxilia nos algoritmos de travessias.

2.3.16. Depth First Traversal (Travessia em Profundidade)

int DepthFirstTraversal(**Antena*** antenaHead, **int** antenaID);

Algoritmo de travessia em profundidade (DFT). Explora o Grafo percorrendo todas as Antenas.

2.3.17. Breadth First Traversal

int BreadthFirstTraversal(**Antena*** antenaHead, **int** antenaID);

Algoritmo de travessia em largura (BFS). Explora o Grafo percorrendo todas as Antenas.

2.3.18. Todos Caminhos Possíveis

```
int TodosCaminhosPossiveis(Antena* antenaHead, int origemID, int destinoID, int  
    distanciaPercorrida);
```

Algoritmo que percorre todos os caminhos possíveis entre duas Antenas referenciadas pelos seus identificadores, também medindo a distância dos diferentes percursos.

3. Implementação

3.1. Percurso até à solução

O desenvolvimento começou com a definição das estruturas Antena e Aresta. Em seguida, criaram-se as funções de inserção ordenada, ligação entre antenas e leitura dos dados via ficheiro não binário. Com estas bases, foi possível montar um grafo funcional. Para garantir persistência dos dados, adicionaram-se funções de leitura e escrita em ficheiros binários. Por fim, foram implementados os algoritmos de travessia DFS e BFS.

3.2. Algoritmo na Linguagem C

A base do grafo foi construída com funções dedicadas à criação e ligação de Antenas e Arestas. A função CalcularArestas foi um ponto fundamental na criação e ligação entre as Antenas que partilham a mesma frequência. Esta função percorre cada Antena e, se encontrar uma outra Antena com a mesma frequência, calcula a distância e cria uma Aresta. A função InserirAresta assegura que cada Aresta é adicionada de forma ordenada à lista das Arestas. Posteriormente a função LigarAntenasArestas garante a correta ligação entre ambas as listas, ligando as Arestas às respetivas Antenas que as originaram. Assim o grafo está completo.

3.3. Leitura e Escrita em Ficheiros

Foram implementadas duas formas de guardar dados.

3.3.1. Ficheiros Texto

A função `LerFicheiro` permite ler uma matriz com Antenas a partir de um ficheiro de texto, convertendo os caracteres lidos em estruturas `Antena`. O ficheiro representa visualmente a localização das antenas na forma matricial.

```
CCC..A
.A...C
...C..
..A...
C..A..
```

Figura III

Já a função `GravarFicheiro` permite guardar a posição atual das Antenas, num ficheiro de texto, na forma matricial.

A *Figura III* é uma representação gráfica do ficheiro de texto.

3.3.2. Ficheiros Binários

Com o objetivo de guardar e retomar toda a estrutura do grafo, foram criadas as funções `GravarFicheiroBinario` e `LerFicheiroBinario`. Estas funções usam estruturas auxiliares (`AntenaFile` e `ArestasFile`, *Figuras III* e *IV*) que contêm apenas dados simples, sem apontadores, para evitar problemas de acesso inválido à memória ao ler os dados presentes no ficheiro binário.

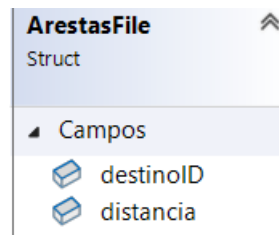


Figura IV

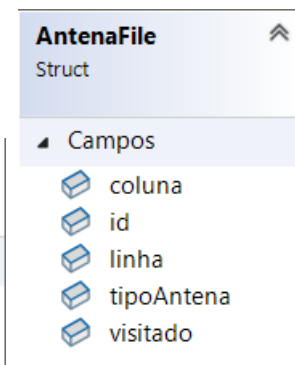


Figura V

Todos os apontadores são tratados durante a leitura, quer seja das Antenas ou das Arestas, com exceção do campo *destino*, apontador da estrutura `Aresta`. Esse apontador só é tratado no final da função `LerFicheiroBinario` com a chamada da função `LigarAntenaDestinoAntenaId`.

3.4. Travessias DFT e BFT

Foram implementados dois algoritmos de travessia.

3.4.1. Depth First Traversal (DFT)

```
137  int DepthFirstTraversal(Antena* antenaHead, int antenaID) {
138
139      if (antenaHead == NULL)
140          return -2;
141
142      Antena* atual = ProcurarAntenaPeloID(antenaHead, antenaID);
143      Arestas* aresta = atual->arestas;
144
145      printf("Estou na antena: %d\n", antenaID);      // debug
146      atual->visitado = 1;
147
148      while (aresta != NULL) {
149
150          if (aresta->destino->visitado == 0) {
151              DepthFirstTraversal(antenaHead, aresta->destinoID);
152          }
153
154          aresta = aresta->proxAresta;
155      }
156
157      return 0;
158  }
```

Figura VI

O algoritmo de travessia Depth First Traversal (DFT) percorre todas as Antenas do Grafo seguindo uma abordagem recursiva. A partir da Antena origem escolhida, visita-se o primeiro vizinho que não tenha sido visitado, ou seja, o campo *visitado* da Antena vizinho está com o valor 0 (Figura VI, linha 150). De seguida a recursividade acontece, isto é, o processo repete-se. Sempre que Antenas sejam visitada, o seu campo *visitado* passa a ter o valor 1 (Figura VI, linha 146).

A recursividade continua enquanto existirem Antenas não visitadas.

3.4.2. Breadth First Traversal (BFT)

```
170  int BreadthFirstTraversal(Antena* antenaHead, int antenaID) {
171
172      if (antenaHead == NULL)
173          return -2;
174
175      int num;
176
177      Antena* atual = ProcurarAntenaPeloID(antenaHead, antenaID);
178      num = NumeroAntenasMesmaFrequencia(antenaHead, atual);
179      int queue[10];
180      int j = 1;
181
182      queue[0] = atual->id;
183
184      atual->visitado = 1;
185
186      for (int i = 0; i < num; i++) {
187
188          printf("Estou na antena: %d\n", atual->id); // debug
189
190          Arestas* aresta = atual->arestas;
191
192          while (aresta != NULL) {
193              if (aresta->destino->visitado == 0) {
194                  queue[j++] = aresta->destino->id;
195                  aresta->destino->visitado = 1;
196              }
197              aresta = aresta->proxAresta;
198          }
199
200          atual = ProcurarAntenaPeloID(antenaHead, queue[j]);
201      }
202
203      return 0;
204  }
```

Figura VII

O algoritmo de travessia Breadth First Traversal (BFT) usa uma queue (Figura VII, linha 179) para navegar pelo Grafo. Este, a partir da Antena origem, processa primeiramente todas as Antenas vizinhas, marcando os respectivos campos *visitados* com o valor 1 (Figura VII, linha 192 à linha 197). Quando os vizinhos da Antena origem esgotarem, o algoritmo vai processar todos os vizinhos do primeiro vizinho da Antena origem.

O processo repete-se até passar por todas as Antenas que estão na *queue*.

Conclusão

Nesta segunda fase do projeto, foi possível consolidar e aplicar os conhecimentos adquiridos sobre a construção e manipulação de grafos dinâmicos utilizando o método de listas ligadas.

A abordagem ao problema permitiu o desenvolvimento de um programa capaz de representar, manipular e percorrer o grafo de forma eficiente e dinâmica. As funções aplicadas de leitura e escrita em ficheiro binário contribuíram para a persistência dos dados, bem como, a reutilização do grafo. Os algoritmos de travessia, BFT e DFT, validam a estrutura do grafo.

Com esta implementação, conclui-se que os grafos dinâmicos são ferramentas poderosas para modelar relações entre elementos, enquanto as listas ligadas oferecem flexibilidade necessária para a manipulação dos elementos.

Link Repositório GitHub:

<https://github.com/a31550/ProjectEDA.git>